Real-Time Semantic Segmentation and Vision-Based Heading Control using PIDNet

Muhammad Tayyab Tahir Qureshi 25280024
Muhammad Abdullah 25280050
Muhammad Ahmad 25280065
Abdul Hafeez 25280098

***Abstract*—Deploying autonomous mobile robots in dynamic environments requires solving visual perception and motion control under strict real-time constraints. This report traces the evolution of real-time semantic segmentation architectures — from early fully convolutional networks to modern multi-branch and attention-based designs — and situates these models within our specific problem: a vision-based heading controller operating on a five-class domain. We examine the accuracy–efficiency trade-off across six architectural categories, based on which, we select PIDNet-S as our baseline architecture and justify that choice. Preliminary fine-tuning on a custom robotics dataset of 90 images (70/10/10 train-val-test split) yielded 62.47% mIoU on validation and 53.76% mIoU on the held-out test set.**

The code for this project is available at: https://github.com/ttqureshi/Real-Time-Semantic-Segmentation-and-Vision-based-Heading-Controller.

The weights for the segmentation model are available at: https://github.com/ttqureshi/Real-Time-Semantic-Segmentation-and-Vision-based-Heading-Controller/blob/main/checkpoints/segmentation/pidNet_wieghts.pt

I. Problem Formulation

Our project tackles a real-time robotic perception and control problem that splits into two coupled subproblems: *real-time semantic segmentation* and *vision-based heading control*.

We use an Intel RealSense D455 in RGB-only mode at 1280×720 pixels, 30 FPS[1]. The system outputs (i) a dense pixel-wise semantic map over five safety-relevant classes — *Road*, *Sidewalk*, *Speed Breaker*, *Human*, and *Obstacle* — and (ii) motion commands (linear velocity, yaw, emergency stop) derived geometrically from that map. Our label space is much narrower than general-purpose benchmarks like Cityscapes (19 classes) [1], which directly affects which models are even worth considering.

The hard real-time constraint is that the full perception-and-command pipeline must complete within one frame period ($\approx$33 ms). Any architecture that cannot sustain 30 FPS is out, regardless of accuracy. The two subproblems are tightly coupled: a segmentation error (an obstacle labelled as road) feeds directly into a dangerous control command. We therefore have to co-select the segmentation model against the latency and boundary-quality requirements of the downstream controller.

[1]https://www.realsenseai.com/products/real-sense-depth-camera-d455f/

II. Literature Review

A. Foundational Methods: From Classification to Dense Prediction

Long et al. [2] introduced Fully Convolutional Networks (FCN), adapting classification backbones for dense pixel-level prediction through in-network upsampling and skip connections. FCN established the encoder-decoder structure that all subsequent work inherits, though it ran at only 5–6 FPS with coarse boundaries. Paszke et al. [3] made the first real push toward real-time with ENet (76 FPS), showing embedded deployment was feasible, but at a steep accuracy cost: 58.3% mIoU. These two papers set up the tension we deal with throughout: accuracy and real-time speed genuinely trade off against each other.

B. Efficient Single-Branch Architectures

Early efficiency gains came from redesigning convolutional blocks. ERFNet [4] factorises each 3×3 convolution into sequential 3×1 and 1×3 passes, reaching 68.0% mIoU at 83 FPS, and over 7 FPS even on a Jetson TX1. Liu et al. [5] incorporate CSP blocks and SPPF pooling to hit 80.8% mIoU at 64.3 FPS. The problem with early single-stream designs is structural: the same channel capacity has to serve both fine spatial detail and abstract semantic context. You cannot optimise for both at once, which is what pushed the field toward multi-branch designs.

C. Multi-Branch Architectures

BiSeNet V2 [6] established the standard two-branch setup, a shallow high-resolution Detail Branch paired with a deep Semantic Branch, fused via a Guided Aggregation Layer, reaching 72.6% mIoU at 156 FPS. DDRNet [7] extended this with bilateral cross-scale connections and DAPPM pooling, pushing accuracy to 79.4% mIoU but dropping throughput to 37 FPS. STDC [8] showed that an explicit spatial branch is not strictly necessary if dense skip connections and boundary supervision are used instead, achieving 71.9% mIoU at 250 FPS.

A persistent problem in two-branch fusion is what PIDNet's authors call semantic overshoot: context features can overwhelm boundary signals and wash out edges. PIDNet [9] addresses this by analogy to PID control: a third Derivative branch predicts boundary maps to damp the overshoot, guided into fusion via a Boundary Attention-Guided (BAG) module.

PIDNet-S achieves 78.6% mIoU at 93.2 FPS, the accuracy-speed frontier at the time of its release.

D. Emerging Directions: Attention and Reparameterisation

BEVANet [10] integrates large-kernel separable attention (SDLSKA) into a bilateral framework to approximate Vision Transformer [11] receptive fields cheaply, reaching 81.0% mIoU at 33 FPS. GCNet [12] takes a different route via structural reparameterisation: a multi-path Golden Cudgel Block that collapses into a single 3×3 convolution at inference, yielding 193.3 FPS at 77.3% mIoU.

E. Specialised Formulations

SFNet-N [13] targets nighttime segmentation through an end-to-end framework with a light-enhancement network and multi-scale attention segmentation, achieving 56.9% mIoU on Dark Zurich, though the enhancement component sacrifices real-time throughput. Cheng and Zheng [14] avoid full-frame segmentation entirely, instead processing sparse horizontal scanlines with a temporal memory network at 0.002 s per line. The approach is fast and can detect pedestrians and vehicles from temporal traces, but it is restricted to fixed-depth scanlines and cannot detect objects at arbitrary image positions outside those depths.

F. Segmentation-Informed Robot Control Pipelines

Ginerica et al. [15] provide the most relevant precedent: a Pioneer 3-DX robot uses MobileNetV2 segmentation to populate a 3D occupancy grid fed into a DWA planner and MPC controller, achieving cross-track errors as low as $e_{ct} = 0.04\,\mathrm{m}$. We follow their modular pipeline design, keeping perception and control separate so each is independently debuggable. Their work treats segmentation as a generic upstream module; we go further by co-selecting the architecture against the boundary quality and latency requirements of our specific controller.

III. Model Performance Comparison

Table I summarises the models we reviewed. Cityscapes results are on the test set unless noted. FPS numbers are hardware-dependent and not directly comparable across rows, since GPU platforms and input resolutions vary.

TABLE I
Performance Comparison of Reviewed Segmentation Models

Method	Year	mIoU (%)	FPS
FCN [2]	2015	62.2[a]	∼6
ENet [3]	2016	58.3	76[b]
ERFNet [4]	2018	68.0	83[c]
BiSeNet V2 [6]	2021	72.6	156
STDC1 [8]	2021	71.9	250
DDRNet-23 [7]	2023	79.4	37
CSP-based [5]	2022	80.8	63.4
PIDNet-S [9]	**2023**	**78.6**	**93.2**
GCNet-S [12]	2025	77.3[d]	193.3
BEVANet [10]	2025	81.0[e]	33

[a] PASCAL VOC 2012; not directly comparable to Cityscapes figures.
[b] ENet result at 512×1024 resolution.
[c] ERFNet result at reduced resolution (640×360).
[d] Validation set; without ImageNet pretraining.
[e] Validation set; with ImageNet pretraining.

IV. Baseline Selection

We select PIDNet-S [9] as our baseline. Three reasons.

Boundary quality. PIDNet's dedicated Derivative branch produces sharper class boundaries by explicitly suppressing the overshoot that erodes edges in two-branch predecessors. This matters directly for the heading controller: extracting the road centreline and detecting obstacle extents from a segmentation mask is much more reliable when boundaries are geometrically clean. Models without explicit boundary modelling (BiSeNet V2, DDRNet) introduce systematic geometric error into the downstream controller.

Literature standing. PIDNet-S has been the standard comparison point in real-time segmentation research since 2023. Every subsequent model we reviewed, GCNet and BEVANet included, benchmarks against it. Using it as our baseline places our results in a directly interpretable reference frame.

Reproducibility. The PIDNet codebase is publicly available with pretrained Cityscapes weights, well-maintained, and validated across multiple hardware configurations. This reduces implementation risk when fine-tuning on our five-class domain.

We exclude GCNet and BEVANet: their results have not been independently validated at scale, and their codebases lack the maturity needed for reliable reproduction. ERFNet, while historically relevant, does not offer sufficient accuracy for a safety-critical navigation task. We retain BiSeNet V2 as a fallback: if PIDNet-S cannot sustain real-time throughput under full pipeline load, BiSeNet V2's 156 FPS provides considerable extra margin while remaining a well-validated architecture.

V. Methodology (Segmentation)

A. Dataset Preparation and Split Design

Our dataset is 90 manually annotated images drawn from a robotics driving video, each paired with a dense pixel-wise mask. We treat the task as a six-class problem: one background class and five foreground classes (*road*, *obstacle*, *human*, *sidewalk*, *speed_breaker*). Raw annotation labels were remapped to a contiguous training label space $\{0, \ldots, 5\}$; ambiguous or artefact regions receive an ignore index of 255 and do not contribute to the loss.

All RGB images and masks are standardised at 432×432 resolution. Because the frames come from a continuous video, a random split would allow temporal leakage through near-duplicate adjacent frames. We instead used a block-based temporal split, ensuring validation and test subsets are maximally separated in time while still containing all required semantic classes. Final partition: 70 training / 10 validation / 10 test images. RGB inputs were normalised using ImageNet mean and standard deviation. Boundary supervision targets were

generated by running a Canny edge detector [16] on each label map and dilating the resulting contours with a 4×4 kernel. We used horizontal flipping as the sole data augmentation and disabled multi-scale augmentation to keep optimisation stable on our small dataset.

B. PIDNet-S Architecture

PIDNet-S [9] is built around three cooperating branches that jointly model pixel-level detail, region-level semantics, and boundary information. Following the PID-control analogy: the **I branch** (Integral) is the deep semantic backbone that progressively downsamples the input to accumulate large-receptive-field category context; the **P branch** (Proportional) preserves higher-resolution spatial cues and receives semantically guided refinement from the I branch via Pixel-Attention-Guided Fusion Modules (Pag); and the **D branch** (Derivative) learns an explicit boundary representation that highlights high-frequency class transitions, supervised directly by the dilated Canny boundary map. Multi-scale context is aggregated by the Parallel Aggregation Pyramid Pooling Module (PAPPM). The Light_Bag boundary-aware fusion module weights detail-preserving P features near edges and stable I features in homogeneous regions; its fused output passes through a semantic head to produce the final class predictions. During training the network emits three outputs (auxiliary semantic, final semantic, boundary map); only the final semantic prediction is used at inference.

C. Transfer Learning and Optimisation

We initialised the network from a pretrained road-scene segmentation checkpoint rather than from scratch, giving the model a strong prior for lane appearance, obstacle boundaries, and common traffic-scene structures. Fine-tuning used SGD with momentum; key hyperparameters: batch size 2, initial learning rate 0.002, momentum 0.9, weight decay 5×10^{-4}, 20 epochs with polynomial learning rate decay $\eta = \eta_0(1 - \text{iter}/\text{max_iter})^{0.9}$ [17].

The composite training objective is

$$\mathcal{L} = \mathcal{L}_{\text{sem}} + \mathcal{L}_{\text{bd}} + \mathcal{L}_{\text{sb}}, \tag{1}$$

where $\mathcal{L}_{\text{sem}}$ combines auxiliary P-branch cross-entropy (weight 0.4) and OHEM-weighted cross-entropy [18] on the final prediction (weight 1.0); $\mathcal{L}_{\text{bd}}$ is weighted binary cross-entropy on the predicted boundary map (coefficient 20.0); and $\mathcal{L}_{\text{sb}}$ adds semantic supervision for pixels near confident predicted boundaries, pushing accuracy higher precisely where class confusion is worst.

D. Validation, Model Selection, and Test Evaluation

After each epoch, we upsampled predicted logits to ground-truth resolution and compared them against targets, excluding ignored pixels. We accumulated a confusion matrix over the full validation split and used mean Intersection over Union (mIoU) as our primary selection metric. The best checkpoint came at epoch 11 with a validation mIoU of 0.6247. We then evaluated the selected model on the held-out test split using mIoU, pixel accuracy, mean class accuracy, per-class IoU, precision, recall, and F1-score, supplemented by the visual diagnostics described in the following section.

VI. Camera Calibration and Sensor Setup

A. Calibration Procedure

Accurate geometric reasoning from camera frames requires knowledge of the camera's internal parameters and correction of lens distortion. Following the forward imaging model [19], a scene point $\mathbf{x}_w$ in the world coordinate frame is mapped to pixel coordinates $\mathbf{u} = (u, v)$ through the projection matrix P:

$$\tilde{\mathbf{u}} = P\,\tilde{\mathbf{x}}_w = M_{int}\,M_{ext}\,\tilde{\mathbf{x}}_w, \tag{2}$$

where $\tilde{\cdot}$ denotes homogeneous coordinates, M_{int} is the intrinsic matrix containing the calibration matrix K, and M_{ext} is the extrinsic matrix encoding the camera's position and orientation in the world frame [19]. The calibration matrix is

$$K = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}, \tag{3}$$

where f_x and f_y are the effective focal lengths in pixels in the horizontal and vertical directions, and (o_x, o_y) is the principal point, the pixel coordinates at which the optical axis pierces the image plane [19]. As noted in [19], the linear projection model must be augmented with distortion coefficients $\mathbf{d} = [k_1, k_2, p_1, p_2, k_3]$ to account for radial and tangential lens distortions that arise in real imaging systems.

We estimate K and $\mathbf{d}$ using OpenCV's `calibrateCamera()`, which implements the planar calibration method of Zhang [20]. Rather than requiring a 3D object of known geometry, this method photographs a flat checkerboard pattern from multiple viewpoints. For each view, it computes a homography between the checkerboard plane and the image, then extracts K by solving a linear system of constraints across all views, followed by non-linear Levenberg–Marquardt optimisation that minimises the total reprojection error, the average pixel distance between projected and detected corner locations.

Nine calibration frames were provided with the project dataset, each showing a 9×6 square checkerboard (yielding $8 \times 5 = 40$ inner corner intersections) taped to a flat wall. One frame was excluded due to severe motion blur. Blurry images produce corrupted 2D corner detections that inflate reprojection error and bias the distortion estimate. The remaining eight images span a range of orientations and viewing distances, giving the homography solver enough angular variation to separate f_x, f_y, and (o_x, o_y) reliably.

For each retained image, `findChessboardCorners()` locates the 40 inner corners and `cornerSubPix()` refines each detection to sub-pixel accuracy by iteratively finding the local intensity-gradient zero-crossing within an 11×11 pixel search window. The calibration images were provided digitally without physical measurements, so the checkerboard square

size was set to an arbitrary unit value. This does not affect K or $\mathbf{d}$ — both are scale-invariant with respect to target size. Only the translation components of M_{ext} change with scale, and we do not use those downstream.

B. Calibration Results

All eight retained images yielded successful corner detections (8/8). The estimated calibration matrix is

$$K = \begin{bmatrix} 608.72 & 0 & 656.30 \\ 0 & 599.78 & 363.65 \\ 0 & 0 & 1 \end{bmatrix} \text{ (pixels)}, \tag{4}$$

and the distortion vector is $\mathbf{d} = [-0.0497,\ -0.0030,\ -0.0037,\ 0.0006,\ 0.0439]$. The RMS reprojection error is 0.462 px, with a worst per-image error of 0.099 px, both well below the 1.0 px threshold considered acceptable for geometric vision tasks.

Three properties of K are worth noting. The principal point $(o_x, o_y) = (656.30, 363.65)$ is 16 px from the image centre horizontally and 4 px vertically, consistent with a well-centred lens. The ratio $f_x/f_y = 608.72/599.78 = 1.015$ is within 1.5% of unity; pixels are square and the lens has no anamorphic distortion. The distortion coefficients are small. The dominant term $k_1 = -0.050$ reflects mild barrel distortion typical of a wide-angle sensor, which the undistortion step removes before any geometric computation.

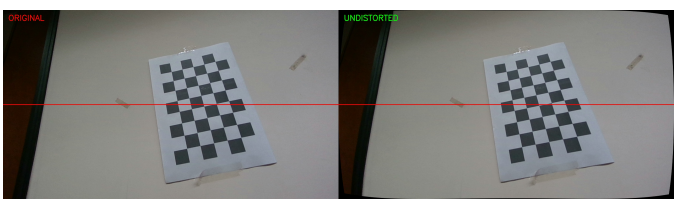

Fig. 1. Side-by-side undistortion validation. The horizontal red reference line confirms that geometric structure is preserved after correction. The crescent-shaped black border on the left edge of the undistorted image is the expected result of correcting the mild barrel distortion ($k_1 = -0.050$).

Figure 1 shows a side-by-side comparison of an original and undistorted frame. At runtime, the heading controller loads K and $\mathbf{d}$ from the saved archive, precomputes a pixel-wise undistortion map once using `getOptimalNewCameraMatrix()` ($\alpha = 0$, cropping to valid pixels) and `initUndistortRectifyMap()`, and applies it per frame via `cv2.remap()` before segmentation inference.

VII. Vision-Based Heading Controller

A. Control Architecture Overview

The vision-based heading controller converts a per-frame semantic segmentation mask and a dense optical flow field into two scalar motion commands: a linear velocity v_{cmd} and an angular (yaw-rate) velocity ω_{cmd}. The pipeline does not requires any depth sensor, inertial measurement unit, or map.

At each frame the following stages execute in order.

1) The undistorted RGB frame is passed to the PIDNet-S segmentation network, which returns a dense class-label mask $\mathcal{S} \in \{0, \ldots, 5\}^{432\times432}$.
2) The current frame and the previous frame are downscaled to half resolution and fed into OpenCV's Farneback dense optical flow estimator producing a two-channel displacement field $\mathbf{F} \in \mathbb{R}^{216\times216\times2}$.
3) The *heading error* e_ψ is computed from $\mathcal{S}$ using the lane-centre estimation described in Section VII-B.
4) A set of *proximity scores* $\{s_{\text{obs}},\ s_{\text{hum}},\ s_{\text{sb}}\} \in [0,1]^3$ is computed from $\mathcal{S}$ and $\mathbf{F}$ using the optical-flow-based depth surrogate described in Section VII-D.
5) ω_{cmd} is computed by a PD heading controller from e_ψ (Section VII-C).
6) v_{cmd} is computed from the proximity scores by a three-tier speed law (Section VII-D).

Segmentation is the bottleneck stage; all subsequent stages complete in under 3 ms per frame on a modern CPU. To sustain 30 FPS, segmentation and optical flow can each be set to run every n frames ($n \geq 1$) and their outputs reused in between.

B. Heading Error Estimation

Reliable heading error requires knowing *where the lane centre lies ahead of the robot*, not merely the robot's current lateral displacement. We therefore sample a *look-ahead band* $\mathcal{B}_{\text{far}}$ defined as the horizontal strip of mask rows $y \in [0.35H,\ 0.55H]$ (roughly 2–4 m ahead at typical road-camera elevation), and extract per-row road-centre estimates from sidewalk boundaries.

a) Per-row road midpoint: For each row y in a band $[y_a, y_b]$, let $\mathcal{L}(y) = \{x : \mathcal{S}(y,x) = 4,\ x < W/2\}$ and $\mathcal{R}(y) = \{x : \mathcal{S}(y,x) = 4,\ x \geq W/2\}$ denote the left and right sidewalk pixel sets. When both are non-empty the per-row road midpoint is

$$m(y) = \tfrac{1}{2}\left(\max_{x\in\mathcal{L}(y)} x + \min_{x\in\mathcal{R}(y)} x\right). \tag{5}$$

This quantity is the midpoint between the inner edge of the left sidewalk and the inner edge of the right sidewalk — the geometric centre of the navigable lane. When one or both sidewalks are absent (e.g. at a junction or due to mis-segmentation), $m(y)$ falls back to the midpoint of the leftmost and rightmost road pixels in that row.

b) Lane centre and heading error: The lane centre estimate is the median of $m(y)$ over all valid rows in $\mathcal{B}_{\text{far}}$:

$$x_{\text{lane}} = \text{median}\{\, m(y) \ :\ y \in \mathcal{B}_{\text{far}},\ m(y) \text{ defined} \,\}. \tag{6}$$

The heading error is the signed horizontal offset of the lane centre from the camera optical axis (principal point o_x from the calibrated K, or $W/2$ in the uncalibrated case):

$$e_\psi = x_{\text{lane}} - o_x. \tag{7}$$

A positive e_ψ means the lane centre lies to the right of the camera axis and the robot must yaw clockwise; a negative e_ψ requires a counter-clockwise correction.

Using the far-band look-ahead instead of a near-band (y close to the bottom of the frame) has two practical advantages. First, on curved roads the far-band correctly shows where the lane is *going*, allowing the controller to anticipate curvature rather than react to it. Second, the far-band samples rows with larger lateral spread between sidewalk edges, reducing the sensitivity of $m(y)$ to single-pixel segmentation noise.

c) Calibrated path: When the camera intrinsic matrix K is available (Section IV), the error is normalised by the focal length f_x to yield a dimensionless angular offset in radians:

$$e_\psi^{\text{cal}} = \frac{x_{\text{lane}} - o_x}{f_x}, \tag{8}$$

which makes the gains K_p, K_d directly interpretable in $\text{rad}\,\text{s}^{-1}\,\text{rad}^{-1}$. This normalised error feeds the calibrated gain pair $(K_{\text{yaw}},\, K_{\text{yaw,d}})$ in place of the pixel-based $(K_p,\, K_d)$.

C. Proportional–Derivative Heading Controller

The yaw-rate command is generated by a PD law on the heading error:

$$\omega_{\text{cmd}}^* = K_p\, e_\psi(t) + K_d\,\big(e_\psi(t) - e_\psi(t-1)\big), \tag{9}$$

where $e_\psi(t-1)$ is the error from the previous frame, stored in controller state. The finite-difference derivative term $K_d(e_\psi - e_\psi^-)$ provides phase-lead compensation: it attenuates overshoot when the robot is already converging toward the lane centre and amplifies the command when the error is growing (e.g. at the entry of a curve).

The command is clamped to a safe operating range:

$$\omega_{\text{cmd}} = \text{clip}(\omega_{\text{cmd}}^*,\, -\omega_{\max},\, +\omega_{\max}), \tag{10}$$

with $\omega_{\max} = 1.0\,\text{rad}\,\text{s}^{-1}$ by default. Default pixel-space gains are $K_p = 0.003\,\text{rad}\,\text{s}^{-1}\,\text{px}^{-1}$ and $K_d = 0.001\,\text{rad}\,\text{s}^{-1}\,\text{px}^{-1}$; calibrated-path gains are $K_{\text{yaw}} = 1.5\,\text{rad}\,\text{s}^{-1}\,\text{rad}^{-1}$ and $K_{\text{yaw,d}} = 0.4\,\text{rad}\,\text{s}^{-1}\,\text{rad}^{-1}$. All gains are exposed as configuration parameters.

D. Optical-Flow-Based Speed and Emergency Stop Logic

A key novel contribution of this work is the derivation of *depth-surrogate proximity scores* for safety-critical objects using only the monocular optical flow field — without any depth sensor. These scores govern a three-tier speed law that reduces velocity or stops the robot before collisions occur.

1) Depth Surrogate via Optical Flow Magnitude: Under forward ego-motion, the optical flow magnitude of a stationary scene point grows with decreasing distance from the camera under the thin-lens approximation. Concretely, for a point at depth Z moving at camera velocity v, the retinal flow magnitude is approximately $\|\mathbf{f}\| \propto v/Z$. A nearby object therefore produces larger flow magnitudes inside its segmentation mask than a distant one of the same class.

We exploit this relationship to define a *road-relative proximity score* for each safety-critical class $c \in \{\text{obstacle, human, speed breaker}\}$.

Let $\mathbf{F}_c$ denote the set of optical-flow vectors at pixels belonging to class c in the flow-resolution mask $\hat{\mathcal{S}} = \text{resize}(\mathcal{S},\, 432{\times}432)$:

$$\mu_c = \frac{1}{|\mathbf{F}_c|} \sum_{\mathbf{f}\in\mathbf{F}_c} \|\mathbf{f}\|_2. \tag{11}$$

The road region ($c = \text{road}$) provides a self-calibrating reference depth: μ_{road} reflects the ego-motion flow at the road surface, which at any given speed is approximately constant across the frame. The road-relative proximity score is

$$s_c = \min\left(\frac{\mu_{\text{road}}}{\mu_c + \varepsilon},\, 1\right), \tag{12}$$

where $\varepsilon = 10^{-4}$ prevents division by zero. The interpretation is direct: if object c produces the same mean flow as the road ($\mu_c = \mu_{\text{road}}$), the score $s_c = 1$ indicates the object is at approximately the same depth as the road surface, i.e. it is far enough away to be safe. If $\mu_c = 2\,\mu_{\text{road}}$, the object is producing twice the road flow, implying roughly half the distance: $s_c = 0.5$ (caution zone). If $\mu_c = 4\,\mu_{\text{road}}$, the object is very close: $s_c = 0.25$ (emergency threshold).

This normalisation is self-calibrating: it requires no fixed depth threshold, no camera height measurement, and no dedicated range sensor. The road itself acts as the depth reference, and the ratio remains meaningful regardless of the robot's absolute speed.

Scores are temporally smoothed with an exponential moving average over a window of $W_s = 5$ frames to suppress flickering caused by frame-level segmentation noise:

$$\bar{s}_c(t) = \frac{1}{W_s} \sum_{k=0}^{W_s-1} s_c(t-k). \tag{13}$$

When class c has no pixels in the current mask (i.e. the object is not visible), s_c is assigned its maximum safe value of 1.0.

2) Three-Tier Velocity Law: The smoothed scores $\bar{s}_c$ drive a priority-ordered velocity decision. Let v_0 be the nominal speed, τ_e the emergency threshold, and τ_c the caution threshold ($\tau_e < \tau_c \le 1$).

a) Tier 1 — Emergency stop (highest priority).: If any human or obstacle score falls below τ_e, the object is critically close and the robot halts immediately:

$$v_{\text{cmd}} = 0 \quad \text{if} \quad \bar{s}_{\text{hum}} < \tau_e \text{ or } \bar{s}_{\text{obs}} < \tau_e. \tag{14}$$

b) Tier 2 — Caution braking: If a human or obstacle score is below τ_c but above τ_e, speed is reduced by a configurable brake factor $\beta \in (0, 1)$:

$$v_{\text{cmd}} = \beta\, v_0 \quad \text{if} \quad \tau_e \le \bar{s}_{\text{hum}} < \tau_c \text{ or } \tau_e \le \bar{s}_{\text{obs}} < \tau_c. \tag{15}$$

c) Tier 3 — Speed-breaker deceleration (lower priority): If the robot is not already braking due to Tier 1 or Tier 2, the speed-breaker score modulates speed independently:

$$v_{\mathrm{cmd}} = \begin{cases} \min(v_{\mathrm{cmd}},\ \gamma_c\, v_0) & \text{if } \tau_e \le \bar{s}_{\mathrm{sb}} < \tau_c, \\ \min(v_{\mathrm{cmd}},\ \gamma_e\, v_0) & \text{if } \bar{s}_{\mathrm{sb}} < \tau_e, \end{cases} \tag{16}$$

where γ_c and γ_e are the speed-breaker slow and crawl factors respectively ($\gamma_e < \gamma_c < 1$).

Default parameter values are $v_0 = 0.5\,\mathrm{m\,s^{-1}}$, $\tau_e = 0.25$, $\tau_c = 0.50$, $\beta = 0.5$, $\gamma_c = 0.7$, $\gamma_e = 0.2$. The final command pair $(\omega_{\mathrm{cmd}}, v_{\mathrm{cmd}})$ is returned for each frame and also logged to a CSV file together with the raw scores, e_ψ, and per-stage processing times.

VIII. System Integration and Video Pipeline

The mathematical pipeline summarised in the *Control Architecture Overview* subsection (*Vision-Based Heading Controller*) is realised as a modular Python package with a thin orchestration layer. This section records how software components connect, in what order each frame is transformed, and where artefacts are written end-to-end.

A. Runtime Structure and Entry Points

The implementation separates *learned perception* (PIDNet inference) from *geometric control* (heading PD law and optical-flow braking). The public PIDNet repository is vendored under `third_party/PIDNet` so that model construction matches training; our fine-tuned weights and YAML configuration (`TEST.IMAGE_SIZE` 432×432, six classes) live under `checkpoints/` and `src/segmentation/configs/`. The script `run_full_pipeline.py` loads the checkpoint with PyTorch, selects CUDA when available, and wraps forward inference in a per-frame function that maps a BGR frame to an $H \times W$ integer label map by bilinear upsampling of logits to the native video resolution. The heading controller itself resides in `src/heading_controller/`; its function `run_pipeline` owns the video I/O loop, optional undistortion, mask acquisition, flow, command synthesis, rendering, and logging. Passing segmentation through a callback rather than baking PIDNet into the controller preserves the same modular boundary emphasised in our literature discussion [15]: the planner can be swapped or unit-tested with stored masks. A Jupyter notebook (`notebooks/main.ipynb`) mirrors the same hook pattern for interactive experiments.

B. Video Capture, Calibration, and Stage Ordering

Input video is read with OpenCV's `VideoCapture`. When a calibration archive is supplied (camera matrix K and distortion coefficients, stored as NumPy arrays), `getOptimalNewCameraMatrix` and `initUndistortRectifyMap` run once at startup; each frame is then undistorted with `cv2.remap` *before* any other processing, matching the projection model in *Camera Calibration and Sensor Setup*. The refined intrinsics K' feed the normalised heading error in Section VII-B when calibration is active.

For every timestamp the software executes the same ordered stages already listed conceptually in *Vision-Based Heading Controller*: (i) segmentation mask (PIDNet or reused from the previous frame if subsampled); (ii) half-resolution greyscale optical flow (Farneback) between consecutive frames; (iii) proximity scores from $\mathcal{S}$ and $\mathbf{F}$ (Section VII-D); (iv) v_{cmd} and ω_{cmd} via braking logic and the PD controller (Sections VII-D and VII-C); (v) overlay rendering and CSV accumulation. Two integer knobs reduce average latency when full 30 FPS model throughput is not required: `seg_every` repeats the last mask for intermediate frames (trading temporal freshness for speed), and `flow_every` reuses the previous flow field between Farneback updates. Default settings run both stages every frame.

C. Outputs and Instrumentation

Unless overridden, batch runs write to `results/full_pipeline_run/`: a per-frame CSV containing v_{cmd}, ω_{cmd}, e_ψ, smoothed proximity scores, a discrete motion-state label, flags for mask reuse, and wall-clock processing time; and an annotated MP4 with segmentation overlay, lane cues, and the velocity gauges described in the implementation. Optional switches dump raw label maps (`.npy`) or colourised PNGs for debugging segmentation without re-inference. Per-stage timers printed at shutdown summarise mean milliseconds spent in undistortion, segmentation, flow, control, rendering, and video encode, supporting the hard real-time constraint framed in *Problem Formulation*.

IX. Results and Evaluation

A. Training Dynamics

Figures 2 and 3 show our validation loss and mIoU curves over 20 fine-tuning epochs. The loss drops sharply in the first three epochs as the pretrained weights adapt to our new label space, then plateaus at a stable low level. The mIoU curve mirrors this trend, rising steeply to approximately 0.57 by epoch 5 before a second gain at epoch 8 brings performance to the 0.62 plateau that holds through epoch 20. We attribute the absence of catastrophic overfitting, despite the small dataset, to transfer learning and our single-augmentation training regime.

B. Per-Class Analysis

On the held-out test set, we obtain an overall mIoU of 0.5376 and 0.9739 pixel accuracy. Figure 4 shows per-class IoU on the validation split, and Figure 5 shows the corresponding precision, recall, and F1 scores.

We observe the strongest results for *road* (test IoU 0.9466) and *obstacle* (test IoU 0.9063), both of which are visually dominant and well-represented in our training data. The weakest performance is on *human* and *speed_breaker*, whose low pixel support, visible in the class-support chart (Figure 6), limits the effective signal available during training. The confusion

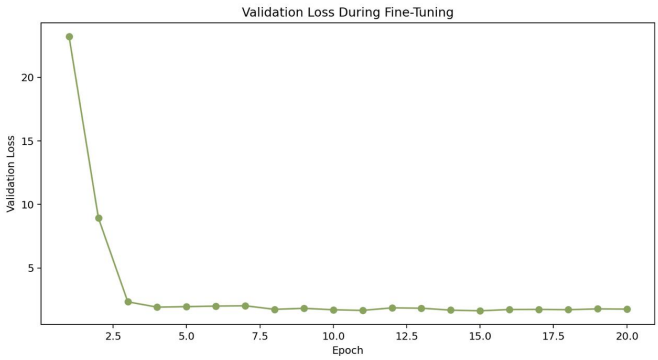

Fig. 2. Validation loss during 20-epoch fine-tuning.

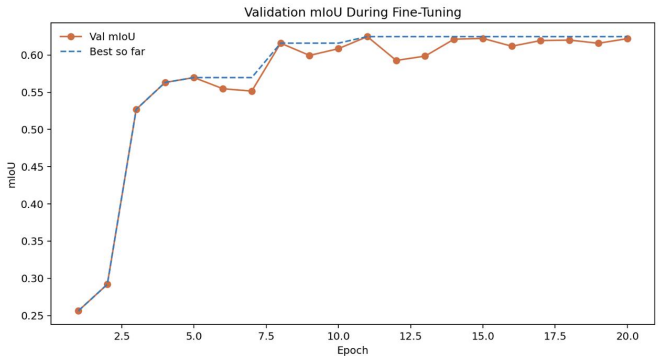

Fig. 3. Validation mIoU during fine-tuning. The dashed line tracks the best checkpoint, achieved at epoch 11 (mIoU = 0.6247).

matrix (Figure 7) confirms that our primary error modes are human pixels being classified as *obstacle* and *speed_breaker* pixels being split between *road* and *obstacle*, the most visually similar pairs in our dataset.

C. Qualitative Evaluation

Figure 8 shows representative test-set frames with ground-truth masks and our model's predictions overlaid as coloured transparencies. Our model produces spatially coherent segmentations with sharp road boundaries and consistent obstacle

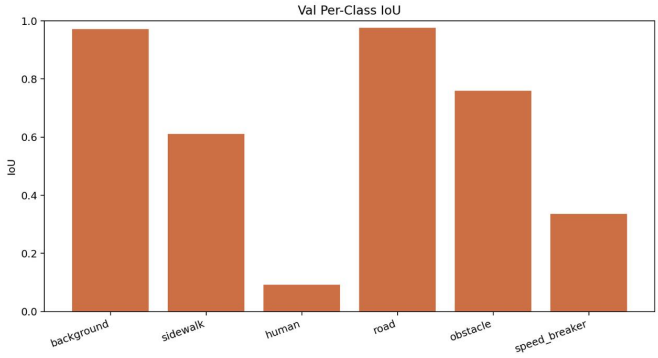

Fig. 4. Validation per-class IoU. *Road* and *background* dominate due to their large pixel support.

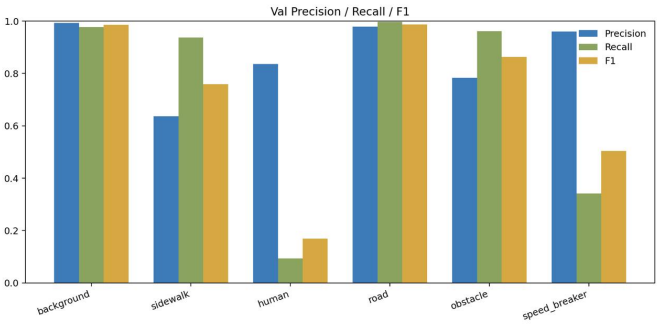

Fig. 5. Validation precision, recall, and F1 per class.

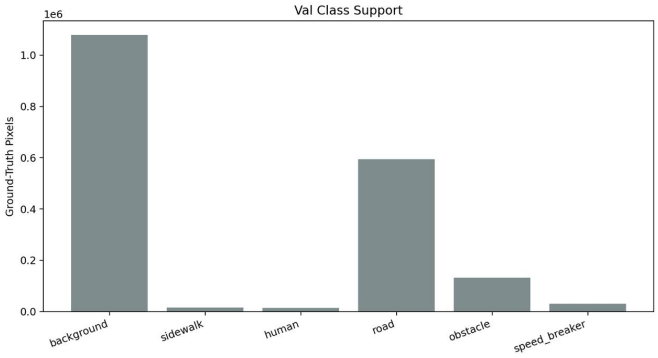

Fig. 6. Validation class support (ground-truth pixel counts). Background and road pixels dominate, while human and speed_breaker are severely underrepresented.

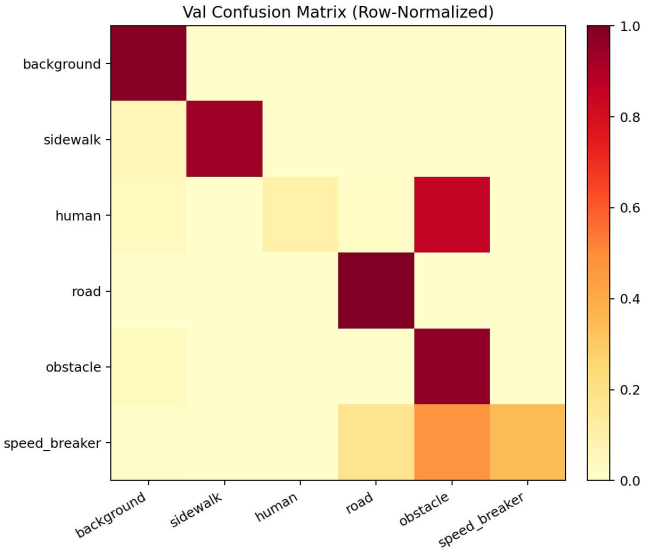

Fig. 7. Row-normalised validation confusion matrix. Strong diagonal entries for *background*, *road*, and *sidewalk*; *human* pixels are frequently misclassified as *obstacle*.

silhouettes across all shown frames. The human silhouette is correctly localised in most frames but occasionally under-segmented at the feet, consistent with the quantitative recall deficit we see in Figure 5.

X. Limitations and Future Work

A. Dataset Scale and Class Imbalance

The most significant constraint on this work is dataset size. Ninety images with a 70/10/10 split yield a test set of ten frames. That means a single misclassified frame shifts test mIoU by roughly 2–3 percentage points. The 8.7-point gap between validation mIoU (0.6247) and test mIoU (0.5376) is most likely a product of high-variance evaluation on a small held-out set, not systematic overfitting, though at this scale the two effects cannot be cleanly separated.

Severe class imbalance compounds this. As shown in Fig. 6, *human* and *speed_breaker* pixels make up a small fraction of the total ground-truth annotation, leaving the model with insufficient gradient signal to learn reliable representations for these classes. The effect shows up directly in the numbers: both classes have the lowest per-class IoU and recall in the evaluation. For a safety-critical navigation task, that is the worst place to be wrong.

The clearest path forward is more data. Targeted annotation of sequences containing humans and speed breakers, combined with class-balanced sampling or focal loss [21] to up-weight underrepresented classes, would address both problems. Expanding to 300–500 images would make the evaluation statistically meaningful and is feasible within a single annotation session.

B. Environmental Robustness

Our model was trained and evaluated under consistent daytime lighting. Wang et al. [13] showed that segmentation performance drops substantially under low-light, rain, or heavy shadow, all directly relevant to an outdoor robot. The optical flow estimator has a related problem: motion blur from rapid manoeuvres at higher speeds produces unreliable flow fields.

Future data collection should cover multiple illumination conditions. Whether the five-class domain needs a dedicated light-enhancement preprocessing stage, or whether synthetic brightness augmentation is sufficient, is an open question worth testing empirically.

C. Resolution and Inference Latency at Full Pipeline Load

All segmentation benchmarks were run on the standalone network. The end-to-end wall-clock latency of the full pipeline — segmentation, optical flow, heading controller, and logging together — was not profiled under worst-case conditions. PIDNet-S comfortably exceeds 30 FPS in isolation at 432×432 input resolution, but the Farneback optical flow computation on a CPU adds non-trivial overhead at higher resolutions. Whether the combined system meets the 33 ms hard constraint on the target embedded platform has not been verified.

D. Model Compression for Embedded Deployment

PIDNet-S was selected in part for its speed, but all benchmarks in this work were measured on a desktop GPU. Deploying the full pipeline on a resource-constrained embedded platform — a Jetson Orin or equivalent — introduces a new bottleneck: PIDNet-S carries approximately 7.0 M parameters and requires around 25 GFLOPs per forward pass at 1024×2048 resolution, which is non-trivial for a device with limited memory bandwidth and no discrete GPU. Three model compression techniques offer practical paths to reducing this overhead without retraining from scratch.

Knowledge distillation [22] trains a compact *student* network to mimic the output distribution of a larger, more accurate *teacher*. Applied to our setting, a smaller single-branch student could be trained to reproduce PIDNet-S's soft class probability outputs on our five-class domain, potentially recovering accuracy that a student trained from scratch would not achieve on only 90 images. This technique is already used in the segmentation literature: SCTNet [12] uses a SegFormer transformer as a teacher to improve a fast single-branch CNN student, demonstrating that the gap between a large teacher and a compact student can be made small even when the architectures differ substantially.

Structured pruning removes entire filters or channels whose contribution to the output is small, reducing both parameter count and the number of FLOPs executed at runtime. BiSeNet V2 [6] explicitly notes channel pruning as one of the two principal acceleration strategies used in prior lightweight networks. For PIDNet specifically, the Derivative branch is the shallowest of the three and therefore the most amenable to pruning.

XI. Conclusion

This report presented the design, implementation, and preliminary evaluation of a real-time robotic perception and control system that integrates semantic segmentation with a vision-based heading controller. We addressed two coupled subproblems: producing per-pixel semantic predictions at 30 FPS or above, and converting those predictions into safe linear and angular velocity commands from monocular RGB input alone.

Our literature review across six architectural generations, from fully convolutional networks to multi-branch and attention-based designs, showed that PIDNet's three-branch design currently navigates the accuracy-efficiency trade-off better than its alternatives. The dedicated boundary branch fixes the overshoot problem that limits two-branch predecessors, and 93.2 FPS on a desktop GPU leaves substantial margin above our hard real-time constraint. We selected PIDNet-S on three grounds: boundary quality, literature standing, and reproducibility.

Fine-tuning on our 90-image, five-class dataset yielded validation mIoU of 62.47% and test mIoU of 53.76%. Performance on the dominant classes was strong: *road* at 0.9466 test IoU, *obstacle* at 0.9063. The failure modes are predictable: low

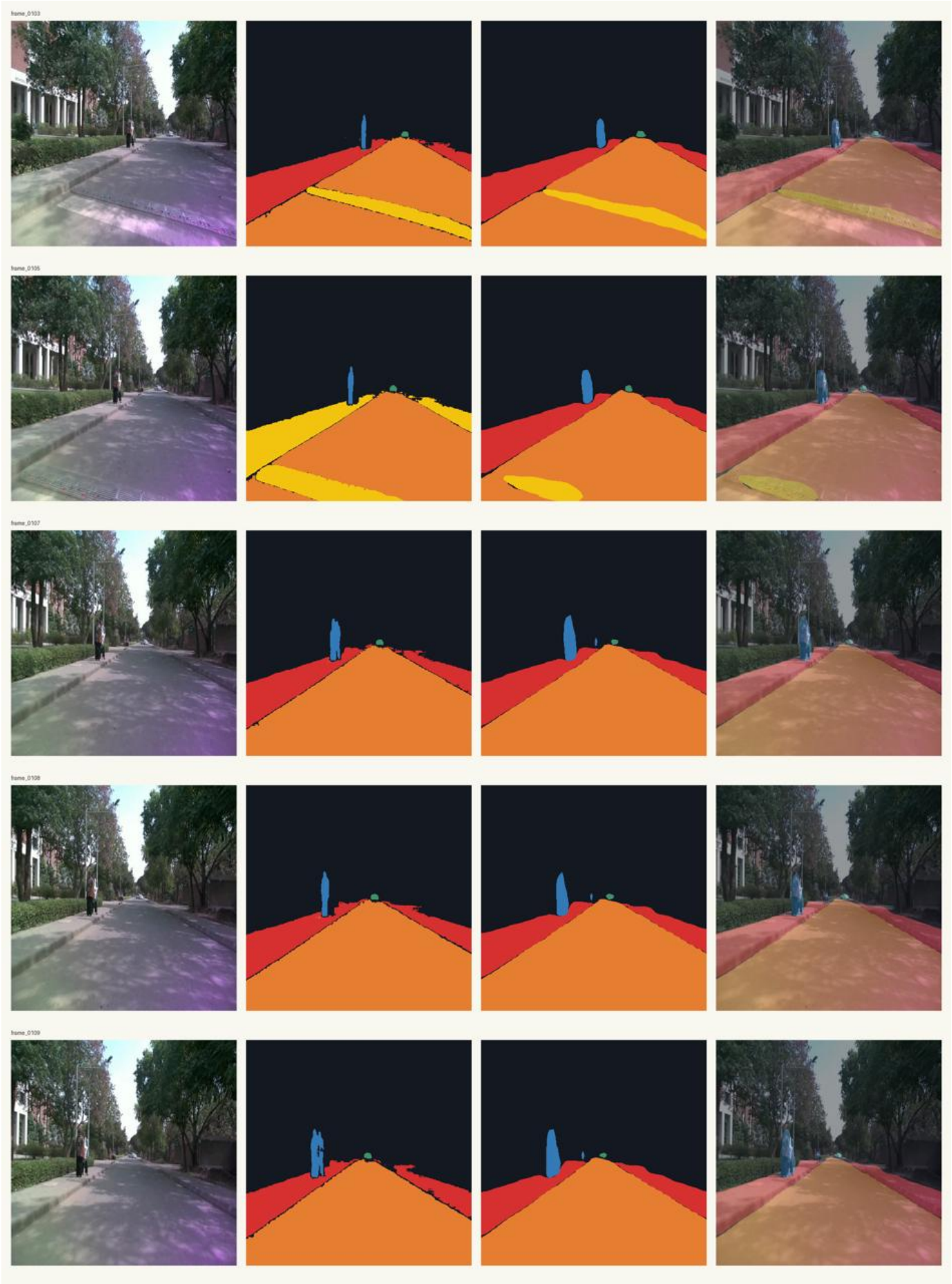

Fig. 8. Qualitative results on test frames. Each row shows (left to right): input RGB image, ground-truth mask, predicted mask, and overlay. Orange = road, red = sidewalk, yellow = speed breaker, blue = human, teal = obstacle.

recall on *human* and *speed_breaker* is a direct consequence of class imbalance in a small dataset.

The heading controller extends the modular pipeline of Ginerica et al. [15] in two ways. A look-ahead band sampling strategy estimates lane centre from far-field rows, letting the controller anticipate curvature rather than react to it. A depth-surrogate proximity score derived from road-normalised optical flow provides a self-calibrating mechanism for proximity estimation of safety-critical objects, generating emergency stop and braking commands with no depth sensor required. This normalisation needs no fixed depth threshold and stays valid regardless of the robot's absolute speed.

The dataset scale is the main limitation. Ten test frames are not enough to draw firm conclusions about a five-class safety-critical system, and the 8.7-point mIoU gap reflects that directly. More annotation, especially for *human* and *speed_breaker*, and closed-loop physical trials on the target robot are the two most important next steps. Overall, the architecture and control pipeline are reliable, interpretable, and modular starting point for safe autonomous navigation in structured outdoor environments.

References

[1] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The Cityscapes Dataset for Semantic Urban Scene Understanding," in *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[2] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3431–3440.

[3] A. Paszke, A. Chaurasia, S. Kim, and E. Culurciello, "ENet: A deep neural network architecture for real-time semantic segmentation," *arXiv preprint arXiv:1606.02147*, 2016.

[4] E. Romera, J. M. Álvarez, L. M. Bergasa, and R. Arroyo, "ERFNet: Efficient residual factorized ConvNet for real-time semantic segmentation," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 1, pp. 263–272, 2018.

[5] L. Liu, L. Zhou, Z. Bing, R. Wang, and A. Knoll, "Real-time semantic segmentation in traffic scene based on cross stage partial block," in *IEEE International Conference on Smart Computing (SMARTCOMP)*. IEEE, 2022.

[6] C. Yu, C. Gao, J. Wang, G. Yu, C. Shen, and N. Sang, "BiSeNet V2: Bilateral network with guided aggregation for real-time semantic segmentation," *International Journal of Computer Vision*, vol. 129, no. 11, pp. 3051–3068, 2021.

[7] Y. Hong, H. Pan, W. Sun, and Y. Jia, "Deep dual-resolution networks for real-time and accurate semantic segmentation of road scenes," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 6, pp. 7348–7361, 2023.

[8] M. Fan, S. Lai, J. Huang, X. Wei, Z. Chai, J. Luo, and X. Wei, "Rethinking BiSeNet for real-time semantic segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 9716–9725.

[9] J. Xu, Z. Xiong, and S. P. Bhattacharyya, "PIDNet: A real-time semantic segmentation network inspired by PID controllers," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 19 529–19 539.

[10] P.-M. Huang, I.-T. Chao, P.-C. Huang, J.-W. Liao, and Y.-Y. Chuang, "Bevanet: Bilateral Efficient Visual Attention Network for Real-Time Semantic Segmentation," in *2025 IEEE International Conference on Image Processing (ICIP)*, 2025, pp. 2778–2783.

[11] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=YicbFdNTTy

[12] G. Yang, Y. Wang, D. Shi, and Y. Wang, "Golden Cudgel Network for Real-Time Semantic Segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pp. 25 367–25 376.

[13] H. Wang, Y. Chen, Y. Cai, L. Chen, Y. Li, M. A. Sotelo, and Z. Li, "SFNet-N: An improved SFNet algorithm for semantic segmentation of low-light autonomous driving road scenes," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 11, pp. 21 405–21 420, 2022.

[14] G. Cheng and J. Y. Zheng, "Sequential semantic segmentation of road profiles for path and speed planning," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 12, pp. 23 869–23 882, 2022.

[15] C. Ginerica, V. Isofache, and S. M. Grigorescu, "Vision dynamics: Environment modelling, path planning and control based on semantic segmentation," in *2021 International Aegean Conference on Electrical Machines and Power Electronics (ACEMP) & International Conference on Optimization of Electrical and Electronic Equipment (OPTIM)*. IEEE, 2021, pp. 481–486.

[16] J. Canny, "A Computational Approach to Edge Detection," *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, vol. PAMI-8, no. 6, pp. 679–698, Nov. 1986. [Online]. Available: http://ieeexplore.ieee.org/xpls/abs_all.jsp?arnumber=4767851&tag=1

[17] L. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking Atrous Convolution for Semantic Image Segmentation," *CoRR*, vol. abs/1706.05587, 2017. [Online]. Available: http://arxiv.org/abs/1706.05587

[18] A. Shrivastava, A. Gupta, and R. Girshick, "Training region-based object detectors with online hard example mining," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.

[19] S. Nayar, "Camera Calibration," in *Monograph FPCV-4-1, First Principles of Computer Vision*, Columbia University, New York, Apr 2025.

[20] Z. Zhang, "A Flexible New Technique for Camera Calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.

[21] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[22] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.